

► **SHIPPED — RACQUETIQ IS LIVE ON THE WEB**

A real person can now open a browser, upload a match and read it back · accounts, Stripe checkout and the App Store path all resolve to the same Pro subscription · HTTPS end to end

RacquetIQ — 24-Hour Report

Aug 18 → 19, 2026 · made by Claude · the product stopped being an iPhone app and became a service

12 commits · ~37k lines of code

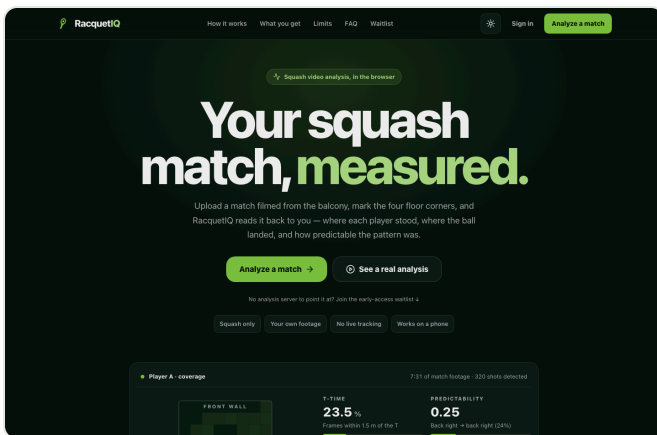
Web app live on Azure

Accounts + billing, verified end to end

Shot precision 41% → 71%

The whole product, in a browser

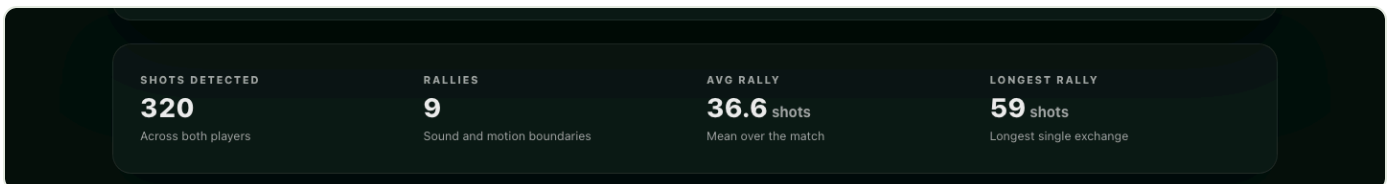
Not the phone app stretched sideways — a separate Vite/React client that shares the data contract, not the components. Upload, four-corner picker, progress, then the read-out. A /demo route renders real pipeline output so the site is never a dead shell.



Live, on a real domain. Deployed to Azure Static Web Apps; the API behind it now terminates TLS through a caddy sidecar, because an HTTPS site cannot call a plain-http server. `8ee065f` `c874d5e`



Real output, not a mockup. 7½ minutes of club squash, the pose skeleton tracking in time with the video, shot markers along the timeline. Same pipeline the app runs. `95f79b7`



What one match yields. 320 shots across both players, 9 rallies, a 59-shot longest exchange — and, below the fold, an honesty panel that states the 63% audited precision rather than hiding it.

Two measurements that changed what got built

41% → 71%

shot-detection precision, against 72 hand-labelled moments across three matches

72.7%

how often "the last player to strike won the rally" is true — an oracle ceiling

651

tests green after the referee gained video, autopilot and three new suites

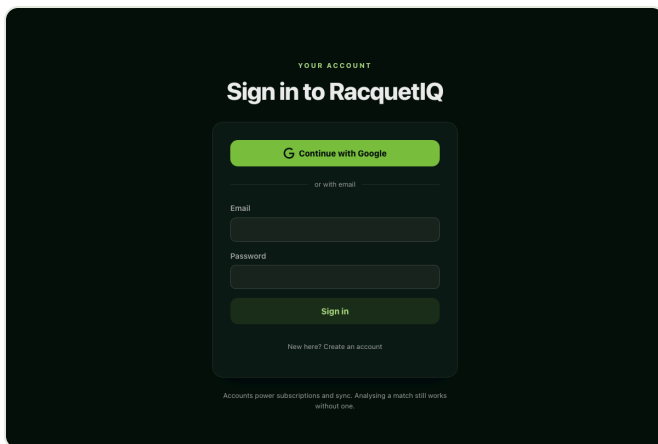
0

auto-confirmed points: at 72.7% the referee always asks, by design

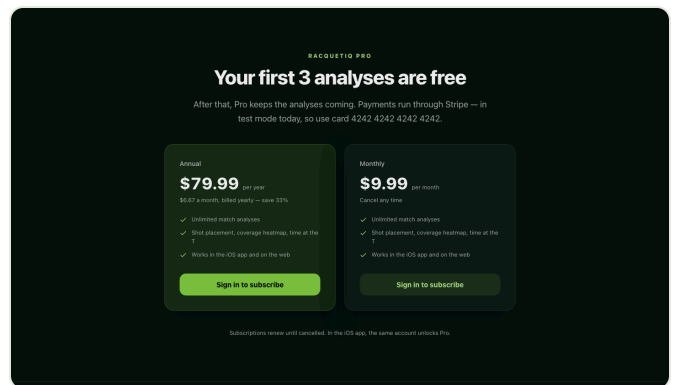
The false positives were never floor bounces. They are real racquet strikes — from the neighbouring courts in these multi-court venues, acoustically identical to true ones (rise 7.23 vs 7.20 dB). Only the video can answer "was a rally being played on *this* court", so a rally-activity gate now runs after the wrist-peak gate. `1bdcc36`

Accounts, billing, and a login that actually works

Supabase for identity, Stripe on the web, the App Store path on iOS — and one entitlement behind all of them. Verified end to end: web signup → test-card checkout → purchase row → the iOS app shows the same Pro; cancel in Stripe and the entitlement is revoked everywhere.



Google leads on the web. Its own GCP project, so the consent screen says RacquetIQ. Apple came *off* the web on purpose — it was configured with the iOS bundle id, which only ever satisfied the native flow, so the button could only have been a dead end. Apple stays what it is: native to iOS. `0855df7`



Three free analyses, then \$9.99/mo or \$79.99/yr. Stripe checkout on the web, RevenueCat on iOS (App Store rules — no Stripe in the app). An admin dashboard behind basic auth reports signups, events and revenue. `32836c6`

What got built in these 24 hours

RacquetIQ on the web — standalone Vite + React + Tailwind client sharing the data contract. Full journey: upload, four-corner picker, progress, read-out. `95f79b7`

Score keeper — courtside PAR-11 scoring that calls the score aloud in squash convention ("eight all", "hand out", "game ball"). Two review findings fixed first: a dropped point from a React batching race, and a week-old match resuming silently as live. `cb66731`

Live referee — camera → Vision pose + streaming audio-onset detection → a rally engine that proposes a winner with a confidence and a plain-English "why". No timeout ever commits by itself. `e3867c9`

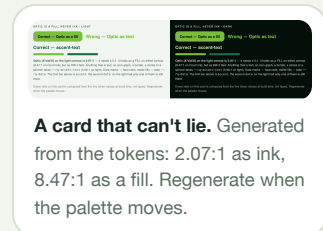
Referee scores videos, and can run on autopilot — one engine, two ways in; a video picker, a result band, and 3 new test suites. `8399679`

Accounts + billing — Supabase profiles/events/purchases all RLS'd, Flask billing blueprint with server-verified confirmation and live subscription re-check, admin metrics. `32836c6`

Hosting, for real — Azure Static Web Apps for the site, a two-container ACI group for the API with caddy terminating TLS, an origin allowlist so the checkout redirect can't become an open redirect. `8ee065f` `c874d5e`

Design pass — two rules this repo had already written down — Optic is a fill, never ink (it measures 1.1:1 on a light surface), and `textFaint` is the disabled tier, not a place to put copy someone has to read. Both now live beside the values in the token files. `e8246e2`

The design system builds itself — every swatch, token value and contrast ratio is read or computed from the token files at build time, after the brand doc was caught quoting ratios for a colour that had moved. `cc375da`



A card that can't lie. Generated from the tokens: 2.07:1 as ink, 8.47:1 as a fill. Regenerate when the palette moves.

LIVE SITE

The web app, deployed and serving

`kind-sea-0e4afca0f.7.azurestaticapps.net`

REPOSITORY

Karanvir1729/RacquetAI — 12 commits in this window

`github.com/Karanvir1729/RacquetAI`

NEXT UP

Wed Aug 19 — the Dixie club sit-down, and a production build for App Review

see the Aug 19 → 20 report